



Sugestão de resolução da parte I

1. ²⁰O paralelismo pode ser explorado a vários níveis: instruções, fios de execução (i.e., *threads*) e processos. **Caracterize** sumariamente cada um destes níveis de paralelismo, salientando as diferenças mais relevantes entre os vários níveis.

Instruções -> as tarefas paralelas são operações básicas (operações aritméticas, acessos à memória, etc), explorado principalmente ao nível do hardware, dentro de cada PU. **Fios de execução** -> explorado ao nível de funções/blocos de código, responsabilidade do programador, explorado principalmente entre PUs. **Processos**, partes de programas ou programas completos, responsabilidade do programador, adequado para exploração de paralelismo entre nós de computação.

2. ¹⁰**Explique** o que é e para que serve o modelo “roofline”.

O **modelo Roofline** é um **modelo analítico de desempenho** usado para **avaliar e prever o desempenho máximo** de aplicações em arquiteturas paralelas. Basicamente diz-nos se vale mais a pena otimizar a computação ou os acessos à memória.

3. ¹⁰**Indique**, justificando, quantos “outputs” diferentes se pode obter com o seguinte programa OpenMP (considere que cada “printf” origina uma linha):

O programa pode produzir **8 outputs diferentes**. O `schedule(static)` fixa quais iterações cada thread executa, e o `barrier` sincroniza as threads a cada iteração, criando 3 passos independentes. Em cada passo, a ordem entre as duas threads é não determinística, resultando em $2^3 = 8$ outputs possíveis.

4. ²⁰As GPUs ganharam nos últimos anos uma grande popularidade. Apesar de assentarem num modelo de memória partilhada apresentam diferenças significativas relativamente às arquiteturas tradicionais (e.g., “multicore”). **Discuta** essas diferenças com base nos seguintes pontos:

- a) ¹⁰Número de fios de execução a especificar e a forma como são escalonados.
- b) ¹⁰Gestão da localidade do dados.

GPUs: Número de threads muito elevado (milhares a milhões de threads); as threads têm pouca sobrecarga pois a gestão é feita essencialmente pelo hardware.

O programador deve explicitamente organizar dados na *shared memory*. Os acessos à memória são muito mais eficientes quando threads de um *warp* acedem a endereços contíguos (localidade especial).

PUs: Número de threads: tipicamente pequeno (ordem das dezenas), geralmente próximo do número de *cores* (ou *hardware threads*). Threads têm uma sobrecarga elevada pois são geridas pelo sistema operativo, o que introduz uma sobrecarga significativa de criação e mudança de contexto.

Localidade é explorada de forma automática pelo hardware. As PUs apostam em caches grandes, multinível e inteligentes para reduzir latência no acesso aos dados.

5. ¹⁰O suporte a novos formatos de vírgula flutuante, designadamente o FP16 e o FP8 permitem aumentar consideravelmente o desempenho dos sistemas de computação. **Explique** como é que estes novos formatos permitem melhorar o desempenho e quais as principais limitações à sua utilização na generalidade das aplicações.

FP16 e FP8 aumentam o desempenho porque permitem mais operações por ciclo, menor tráfego de memória e melhor eficiência energética, mas não substituem o FP32/FP64 pois têm menos precisão na representação restringindo a sua utilização a domínios específicos.